GradePath Final Project Design Document

Course: CS 19300

Project Name: GradePath

Author: Surjosnato Dhar

1.1: The Problem I Want to Solve

College students often struggle to calculate their current grades and determine what scores they need on future assignments or exams to achieve their desired final grades, especially when courses use weighted grading categories. I hate it when I set up a prediction for myself, and it's not even close to the final weighted grade. It makes the attempt at a prediction completely meaningless, even though it could be a valuable way for students to maximize the efficiency of what they're studying for.

1.2: My Idea (The Solution)

I plan to build a Java-based application that lets students enter their course grading categories, weights, and scores to calculate their current and projected grades. The program will also let students test hypothetical scores and determine what they need on remaining assessments to achieve a desired final grade (or it will tell them if the desired grade is no longer possible).

2.1: The Eisenhower Matrix and MVP

	Urgent	**Not Urgent**
Important	**Do - (Your MVP)** ● Create a Java-based interface that allows users to enter grading components and their percentage weights. ● Allow users to enter scores for completed assessments and	**Schedule - (For V2.0+)** ● Save and load course grading information using local files so users don't need to reenter their data ● Support multiple courses and allow users to switch between them.

	identify which assessments remain ungraded. ● Calculate the student's current weighted grade based on completed assessments. ● Allow users to enter hypothetical scores for remaining assessments to calculate their projected final grade. ● Calculate the average score needed across remaining assessments to reach a desired final grade, including identifying when a target is mathematically impossible.	● Allow users to track individual assignments within grading categories rather than entering only overall scores. ● Add customizable letter-grade thresholds to display predicted letter grades alongside numerical percentages.
Not Important	**Delegate - (Use a library)** ● Use Java's built-in Scanner class to handle user input instead of creating a custom input system. ● Use Java's ArrayList collection to store grading components and scores instead of building a custom data structure. ● Use Java's built-in printf() formatting to display grade percentages and calculation results clearly instead of implementing custom formatting logic.	**Delete - (Cut it)** ● Develop a dedicated mobile application for Android or iOS. ● Integrate directly with Purdue Brightspace to automatically retrieve assignment grades. ● Implement AI or machine learning to predict future student performance. ● Add user accounts and online authentication. ● Build a complex dashboard with interactive charts and animations.

2.2: Tech Stack

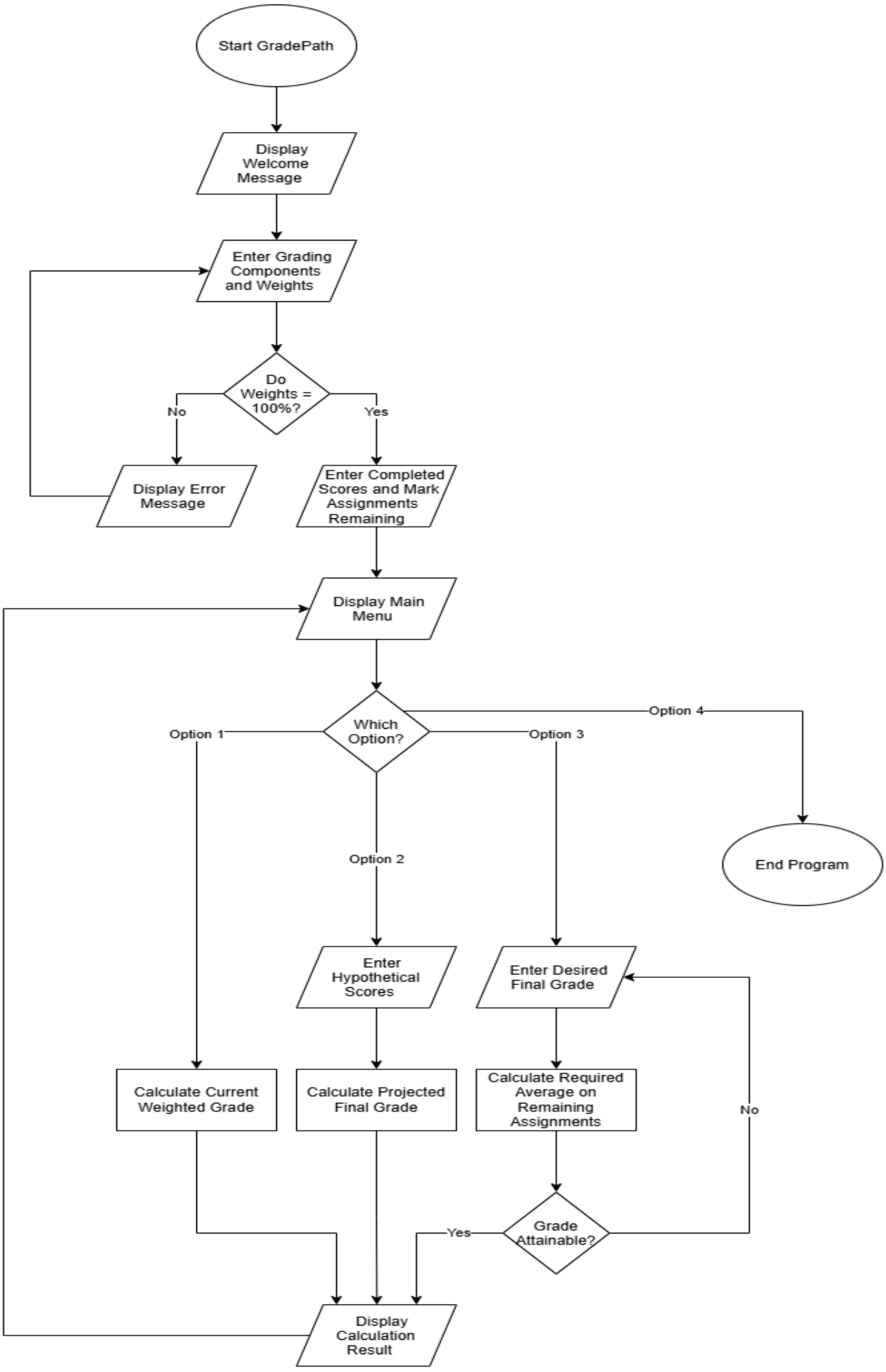
- **Programming Language:** Java, Bash + RegEx.
- **Development Environment:** Visual Studio Code, IntelliJ IDEA, Ed Workspaces.
- **User Interface:** Basic menu with possible dropdowns.
- **Libraries:** Java's built-in classes and libraries.
- **Data Storage:** In-memory using Java objects.
- **External Data Sources / APIs:** None; all input is done manually by the user.
- **Version Control:** Git and GitHub for tracking progress and maintaining the repository.

2.3: Chunked Build Plan

1. **Set up the project repository:** Create the GitHub repository, post the Eisenhower Matrix and the flowchart, and add a basic README and project planning documents.
2. **Build the basic interface:** Create the main Java program with a simple menu allowing users to select grade calculation options or exit.
3. **Implement grading component input:** Allow users to enter grading components like tests, assignments, quizzes, and their percentage weights. Add some kind of check to make sure that it all adds up to 100%.
4. **Implement score entry:** Allow users to enter scores for completed assessments and mark remaining assessments as ungraded.
5. **Calculate current grades:** Implement weighted grade calculations using completed assessments and display the student's current percentage.
6. **Implement hypothetical grade predictions:** Allow users to enter hypothetical scores for remaining assessments and calculate their projected final grade.
7. **Implement target-grade calculations:** Allow users to enter a desired final grade and calculate the average score needed on remaining assessments, including detecting impossible targets.
8. **Test and finalize the MVP:** Test calculations and edge cases, improve error handling, and update the README with accurate setup instructions and examples.

2.4: System Diagram

Flowchart on the page below:


Start GradePath
Display Welcome Message
Enter Grading Components and Weights
Do Weights = 100%?
No
Yes
Display Error Message
Enter Completed Scores and Mark Assignments Remaining
Display Main Menu
Which Option?
Option 1
Option 2
Option 3
Option 4
End Program
Enter Hypothetical Scores
Enter Desired Final Grade
Calculate Current Weighted Grade
Calculate Projected Final Grade
Calculate Required Average on Remaining Assignments
Grade Attainable?
Yes
No
Display Calculation Result

3.1: GitHub Setup and README

Public GitHub Repository:

https://github.com/dharsurjosnato/GradePath

The repository contains the project README, Eisenhower Matrix, system flowchart, and initial project folder structure (all inside the /docs directory).